

# Analysis of the Trading Model's Performance and Issues

## Unrealistic Extreme Predictions and Model Calibration

The model is clearly outputting **unrealistically extreme 20-minute return predictions** ( $\mu$ ). In the backtest log, it frequently predicts spikes of **20–40% in just 20 minutes**, e.g. a 44.25% move for AGNC <sup>1</sup>. Such moves are virtually impossible under normal market conditions. This suggests something went wrong with the model's output calibration or training: - **Scaling or Data Bug**: The predicted returns ( $\mu$ ) might be mis-scaled. For instance, if the model's outputs were meant to be small fractional changes but got interpreted as percentages, it would inflate predictions. A normalization or labeling error could cause the model to output values an order of magnitude too high. - **Overfitting to Noise**: The model may have latched onto random patterns or outliers in training data. If it wasn't regularized or if it **saw spurious correlations**, it could produce wildly exaggerated signals in live data (e.g. predicting huge jumps on minor fluctuations). - **Overconfident Outputs**: The model shows **p\_netpos = 1.000** for every prediction, with **entropy = 0**. In other words, it's entirely sure the market will go up every time. This is a classic sign of an **overconfident, mis-calibrated model** <sup>2</sup>. In reality, no model should be 100% confident on every short-term move – such extreme probability outputs often indicate the model's probability estimates are not trustworthy (likely a result of how the model was trained or an issue with converting model outputs to probabilities).

In summary, the model is **predicting random extreme changes that are impossible** because it's either **mis-calibrated or fundamentally flawed** in how it interprets input data. It consistently predicts large positive returns with near-certainty, which does not reflect real market behavior. This caused the strategy to enter trades on false signals (since those big moves never materialized, the trades mostly ended flat or with slight losses after costs).

## Role of SNR and Entropy in the Strategy

You noticed that **SNR (Signal-to-Noise Ratio)** is still being printed as **1.00** for every trade, despite not influencing decisions. This likely means: - **SNR Not Implemented**: The SNR values are probably placeholders or calculated incorrectly (always yielding 1). It seems the strategy did **not actually use SNR to filter trades**. Perhaps SNR was intended to be  $\mu/\sigma$  or a confidence metric, but if so, it's not being applied (or it's computed incorrectly, e.g. normalized to 1). The output shows no trades with  $\text{SNR} > 1$  because every trade had SNR fixed at 1.00, indicating a coding oversight. - **Entropy Not Effectively Used**: Similarly, the **entropy** measure was supposed to reflect prediction uncertainty (low entropy = high confidence). In theory, incorporating an entropy filter means only acting on low-uncertainty predictions <sup>1</sup> <sup>3</sup>. However, in the log entropy is 0 for all predictions, which made the condition trivially true every time. This suggests the **"prediction + entropy" strategy was not implemented correctly**. Possibly the code set entropy to zero whenever a direction was predicted (since p\_netpos was 1 or 0), thus every signal looked perfectly confident. In practice, we expect a spectrum of entropy values (higher entropy when the predicted

distribution is unsure of direction). The fact it's always zero means the strategy never actually filtered out noisy signals – it treated every prediction as high-confidence.

**Bottom line:** The strategy as coded is effectively just “always trust the model's direction,” because SNR and entropy filters did nothing. Given the model's issues (always predicting upward with extreme confidence), the strategy ended up taking nearly every possible trade, including many that were just noise.

## Investigation of the Prediction+Entropy Implementation

It's important to **deeply review how the model's predictions and uncertainties are used:**

- **Distribution vs Deterministic Output:** If the model was supposed to output a distribution (mean  $\mu$  and std  $\sigma$  for returns), then  $p_{netpos}$  should be computed as the probability of a positive return. In a well-calibrated system, we'd see  $p_{netpos}$  vary (e.g. 0.60, 0.75, 0.99) and entropy > 0 for uncertain cases. The fact that  $p_{netpos}$  is pegged at 1.000 suggests either the model's predicted  $\mu$  is so large relative to  $\sigma$  that the probability of a positive outcome rounds to ~100%, or the code simplified  $p_{netpos}$  to a binary 0/1 decision. You should verify that  $p_{netpos} = \Phi(\mu/\sigma)$  (if assuming a normal forecast) and not accidentally hard-coded. It's suspicious that even the smallest predicted  $\mu$  (1.22%) with a non-zero  $\sigma$  resulted in 100% probability – in a correct calc, a 1.22% mean with reasonable  $\sigma$  would not be 100% certain.
- **Entropy Calculation:** Check how entropy is calculated. If it's treating the prediction as a classification with probability =  $p_{netpos}$  for “up” vs  $(1-p_{netpos})$  for “down,” then entropy =  $-[p\log(p) + (1-p)\log(1-p)]$ . With  $p \sim 1.0$ , entropy should be near 0 but not exactly zero unless  $p$  was truly 1.000. The logs show entropy to three decimals as 0.000 consistently. This could be due to rounding, but it's more likely the code is directly setting entropy to 0 whenever a clear prediction is made. **Ensure that the entropy reflects the model's confidence properly.** If the model output is always one-sided, maybe use a softer probability (or introduce dropout for a distribution of predictions) so that you get a meaningful confidence level. Right now, the strategy isn't “seeing” any difference between a strong signal and a borderline one – everything looks equally certain, which is dangerous.

In essence, **the intended strategy (using prediction + entropy) isn't functioning as expected.** The implementation issues made it equivalent to a simplistic always-long strategy on any positive signal. This is why the performance was poor: the model's slight directional edge (if any) was drowned out by overtrading and costs.

## Suggestions for Improvement

To address these problems and improve the strategy, consider the following changes:

1. **Calibrate and Constrain the Model's Outputs:** Adjust the model so that its predicted returns ( $\mu$ ) and probabilities are realistic. This might involve:
2. **Fixing scaling issues** – double-check how you convert model output to a percentage return. Ensure any normalization done in training is properly reversed in inference.
3. **Regularizing extreme predictions** – for example, use a activation function or loss penalty that discourages predicting outlandishly large returns. If historically 99% of 20-minute returns are within  $\pm 1\%$ , the model shouldn't easily predict +30%. Consider training on log-returns or clipping the target distribution to focus on a reasonable range.

4. **Probability calibration** – since the model currently behaves as overconfident, apply techniques like temperature scaling or Platt scaling to the output probabilities <sup>2</sup> . A well-calibrated model will output more moderate confidences (e.g. 60%/40%) when unsure, which will translate into a non-zero entropy and allow the strategy to distinguish high-confidence signals from weak ones.
5. **Incorporate SNR/Uncertainty Filters Properly:** Leverage the signal-to-noise ratio and entropy measures as intended:
6. Define **SNR =  $\mu/\sigma$**  (or another appropriate measure of confidence in the predicted move). Then set a threshold (e.g.  $\text{SNR} > 2$  or  $3$ ) to execute trades. This ensures you trade only when the expected return is several times larger than the predicted uncertainty (i.e. the signal stands well above noise). Currently, because SNR was stuck at 1, no filtering happened. Once fixed, this can prevent trading on marginal predictions.
7. Use the **entropy filter** to avoid noisy periods. For example, require entropy below a certain cutoff (meaning the model is confident in the direction) before taking a position. Empirical research shows entropy-based filtering can reduce false signals and focus on reliable trends <sup>1</sup> . By correctly computing entropy of the prediction, the strategy can **skip trades when the market is too unpredictable** (high entropy) and only act when there is a clear, low-entropy signal.
8. **Adjust Thresholds to Account for Costs:** The simple rule “buy if prediction  $> 0.10\%$ ” was probably too lenient. A  $0.1\%$  predicted gain is just equal to your round-trip spread cost ( $\sim 0.1\%$ ), meaning on average such trades won't profit after fees. It's better to **require a higher expected return to open a trade**, say at least  $> 0.2\%$  or  $0.3\%$ , to have a cushion above transaction costs. In other words, incorporate the trading friction into your decision threshold. This will reduce overtrading and improve net profitability by only trading when the model's edge is strong enough to overcome costs.
9. **Verify Data Integrity and Model Inputs:** Investigate if any anomalies in data could be driving the extreme predictions. For example, check if the model might be reading **post-open spikes or bad ticks** as signals. Ensure that all features are properly scaled and that the model isn't inadvertently using future data (leakage). Sometimes, unrealistic predictions arise from the model encountering a situation outside the training distribution – for instance, if at minute 103 the data had a gap or corporate action that the model misinterpreted. Improving the training dataset (e.g. filtering out outliers or adding examples of stable periods) and retraining can make the model more stable. In short, **make the model more robust to noise** so that it doesn't trigger on random fluctuations.
10. **Refine the Trading Strategy Logic:** Beyond threshold triggers, consider strategy enhancements:
11. **Continuous re-evaluation:** Instead of blindly closing after a fixed 20-minute window, allow some flexibility. For example, if the model still predicts a further rise at the 20-minute mark, you could **hold the position longer (or re-enter immediately)** rather than closing and reopening (which incurs extra spread cost). This way, you ride the trend as long as the model indicates strength, and exit only when the prediction turns neutral or negative. It reduces unnecessary round trips.
12. **Short selling on negative signals:** If you modify the model to output negative predictions as well (once calibration is fixed, you should occasionally get  $\mu < 0$  for expected drops), you can profit from those by taking short positions. This doubles the opportunity (the model currently missed any

chance to short because it never signaled down moves). Of course, apply similar confidence thresholds for shorts (e.g. prediction < -0.2% and low entropy).

13. **Risk management:** Implement stop-loss or take-profit rules based on the model's uncertainty. For instance, if a trade goes significantly against the prediction (suggesting the model was wrong this time), cut it quickly to avoid large drawdowns. Conversely, if a trade quickly moves in your favor, you might secure some profit or at least adjust the stop to breakeven. This wasn't addressed in the initial strategy but can improve outcomes when the model occasionally makes a bad call.
14. **Reduce Overtrading and Focus on Quality Signals:** The backtest took 52 trades in a short window, which is a lot. Overtrading can amplify costs and slippage. Consider **trading less frequently by picking only the top signals**. For example, instead of opening positions in every stock that crosses a 0.1% threshold, you might rank predictions at each time step and only take the highest-confidence prediction (or top 2–3) at once. This ensures capital is allocated to the best opportunities and avoids a scattergun approach. By filtering for stronger signals (using SNR/entropy as mentioned), you'll automatically cut down the number of trivial trades and focus on those more likely to succeed <sup>3</sup>.

By implementing these improvements, the strategy should become more realistic and selective. In summary, **fix the model's output calibration and confidence measures first** – this is crucial so that predictions make sense. Then, **use those confidence metrics (SNR, entropy) to guide trading decisions**, so you only act on truly significant signals. Finally, **fine-tune the trade execution rules** to avoid friction and let profits run when conditions are right. These steps will address what went wrong (unreliable model outputs and lack of filtering) and move the strategy closer to its intended performance.

---

<sup>1</sup> <sup>3</sup> Optimizing Algorithmic Trading with Machine Learning and Entropy-Based Decision Making[v2] | Preprints.org

<https://www.preprints.org/manuscript/202502.1717/v2>

<sup>2</sup> Model Calibration | TDS Archive

<https://medium.com/data-science/model-calibration-explained-a-visual-guide-with-code-examples-for-beginners-55f368baf72>